

Section A: Deploying Flask Applications with Gunicorn (10 Questions)

1. What is Gunicorn, and why is it used in Flask deployment?

Gunicorn (Green Unicorn) is a WSGI HTTP server for running Python web apps in production. Flask's built-in server is only for development, while Gunicorn handles multiple requests, scaling, and stability.

2. Write the command to run a Flask app using Gunicorn with 4 worker processes.

```
gunicorn -w 4 app:app
```

3. Difference between flask run and running via Gunicorn.

- flask run → development server (debugging, single process).
- Gunicorn → production server (multi-process, better performance, stable)

4. How do you integrate Gunicorn with **Nginx** for production deployment?

Nginx acts as a reverse proxy. Nginx handles client requests, static files, and forwards API requests to Gunicorn (running Flask). This improves performance and security.

5. Debug: Gunicorn throws `ModuleNotFoundError: No module named 'app'`. What might be the cause?

This happens if the file or app name is wrong. Example: If the file is `main.py`, use `gunicorn main:app`.

6. How do you enable logging in Gunicorn?

```
gunicorn app:app --access-logfile access.log --error-logfile error.log
```

7. What is the default worker type in Gunicorn, and when would you change it?

It's sync workers (handle one request per worker at a time). Change to async (gevent, eventlet) when handling long-running or many concurrent requests.

8. Explain the difference between sync and async workers in Gunicorn.

- Sync → good for CPU-bound apps, simple.
- Async → better for I/O heavy tasks (like APIs with many open connections).

9. Write a sample Gunicorn config file with settings for workers, timeout, and log level.

```
workers = 4
```

```
timeout = 120
```

```
loglevel = "debug"
```

10. Compare Gunicorn vs uWSGI for Flask deployments.

- Gunicorn → simpler, Python-focused, easier to set up.
- uWSGI → more configurable, supports multiple languages, but more complex.
For most Flask apps, Gunicorn is enough.

Section B: Introduction to Microservices Architecture (15 Questions)

11. Define microservices architecture in your own words.

Microservices architecture splits an application into small, independent services, each handling one responsibility (e.g., user service, order service).

12. Compare Monolithic vs Microservices architecture with examples.

- Monolith → all features in one big codebase (e.g., single Django app).
- Microservices → multiple smaller apps connected by APIs (e.g., separate student and teacher services).

13. What are the advantages of microservices over monoliths?

Scalability, independent deployment, easier debugging, technology flexibility, and fault isolation.

14. Explain the concept of **service registry and discovery** in microservices.

It's a system where services register themselves (like a phonebook) so others can find them dynamically (e.g., using Consul, Eureka, or Kubernetes DNS).

15. How do microservices communicate with each other? Give 3 methods.

- REST APIs (HTTP).
- Message queues (RabbitMQ, Kafka).
- gRPC (fast binary communication).

16. What is an API Gateway in microservices, and why is it required?

It acts as a single entry point for clients. It routes requests to services, handles authentication, caching, and load balancing.

17. How do you ensure **fault tolerance** in microservices?

Use retries, circuit breaker pattern, load balancing, and redundancy to ensure services don't crash the whole system.

18. What are the main challenges in moving from monolithic to microservices?

Service communication, data consistency, monitoring, deployment complexity, and team coordination.

19. Explain the concept of **bounded context** in domain-driven design for microservices.

Each service should only handle its domain boundary. Example: Student service handles students only, Teacher service handles teachers only.

20. What role does Docker play in microservices?

Docker packages each service in a container with dependencies. This makes deployment consistent across machines.

21. Compare synchronous vs asynchronous communication between microservices.

- Sync → request/response (REST).
- Async → event/message-driven (RabbitMQ/Kafka). Async is better for scalability.

22. What is the **circuit breaker pattern** in microservices?

It stops sending requests to a failing service temporarily and returns fallback responses. Prevents cascading failures.

23. Which database strategy fits microservices better: one DB per service or shared DB? Why?

Best practice → one DB per service. It keeps services independent. Shared DB creates tight coupling.

24. Explain how **service orchestration** differs from **service choreography**.

- Orchestration → a central service controls workflows.
- Choreography → services coordinate themselves by sending events.

25. Give a real-world example (like Netflix, Uber, or Amazon) where microservices architecture is applied and why.

Netflix uses microservices (thousands of services). Example: Login service, Recommendation service, Streaming service. This allows them to scale globally.

Section C: Building Microservices with Python (15 Questions)

26. What Flask or FastAPI feature makes them good for microservices?

- Flask → lightweight, simple routing.
- FastAPI → async support, automatic docs (Swagger). Both good for REST APIs.

27. Build a simple Flask microservice that returns the current server time as JSON.

```
from flask import Flask, jsonify
import datetime
app = Flask(__name__)
@app.route('/time')
def get_time():
    return jsonify({"time": str(datetime.datetime.now())})
```

28. How do you make two Python microservices talk to each other via REST? Give a coding example.

```
import requests
response = requests.get("http://service-b:5000/data")
```

29. What libraries can you use in Python to implement message queues between microservices?

Celery (with RabbitMQ/Redis), Kafka-Python., Kombu.

30. Write a FastAPI service that accepts POST data { "name": "John" } and returns "Hello John".

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/hello")
def hello(data: dict):
    return {"msg": f'Hello {data['name']}'}
```

31. How would you secure communication between Python microservices?

Use HTTPS, API keys, JWT tokens, and service-to-service authentication.

32. Explain how you can scale Python microservices horizontally.

Run multiple containers (using Docker/Kubernetes) behind a load balancer.

33. Debug: One microservice is working locally but fails inside Docker. What should you check first?

Check network configuration, environment variables, ports, and DB connection URLs.

34. What is **JWT** and how would you use it for authenticating Python microservices?

JWT (JSON Web Token) is used for authentication. One service issues tokens, and others validate them for secure communication.

35. How do you monitor health checks in microservices using Python?

Create an endpoint /health returning status. Use monitoring tools like Prometheus + Grafana.

36. Write a Python microservice that connects to MongoDB and retrieves all users.

```
from flask import Flask, jsonify

from pymongo import MongoClient

app = Flask(__name__)

client = MongoClient("mongodb://localhost:27017/")

db = client['school']
```

```
@app.route('/users')
def get_users():
    return jsonify(list(db.users.find({}, {"_id": 0})))
```

37. Explain the role of **Celery** in Python microservices.

Celery handles background tasks (like sending emails) so services don't block API requests.

38. How would you implement **caching** in microservices using Redis with Python?

```
import redis
cache = redis.Redis(host='localhost', port=6379)
cache.set("user:1", "John")
```

39. Compare Flask vs FastAPI for building microservices. Which one is better for high-performance systems?

- Flask → easy, flexible, mature.
 - FastAPI → faster, async, built-in validation & docs.
- For **high performance**, FastAPI is preferred.

40. Design a **school management system** as a microservices architecture with at least 4 services. Identify each service and its responsibility.

 **Student Service** → manages student records.

- Handles student registration, profiles, grades, and attendance.
- Own database with student info.
- APIs: /students, /students/{id}

 **Teacher Service** → manages teachers and subjects.

- Manages teacher details, subjects, and class assignments.
- APIs: /teachers, /teachers/{id}

 **Course Service** → handles courses and assignments.

- Handles subjects, timetables, and course materials.
- Links teachers ↔ students.
- APIs: /courses, /courses/{id}/enroll

 **Auth Service** → manages login, tokens, and roles.

- Provides login, roles (student/teacher/admin), and JWT authentication.
- APIs: /login, /register, /verify-token